DATA STRUCTURE LAB

(ETCCDS202)

LAB FILE SUBMITTED TO

K.R. MANGALAM UNIVERSITY

FOR

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE & ENGINEERING

SPL. ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

Submitted by:

Aryan Solanki
Roll No: 2501730347

Faculty:

Mrs. Neetu Chauhan

Unit 1: Foundations & Algorithmic Analysis

PC-01: Stack ADT (Design + Use)

```python
stack = []
MAX = 5

def push(item):
    if len(stack) < MAX:
        stack.append(item)
        print(f"Pushed {item}")
    else:
        print("Stack Overflow")

def pop():
    if not stack:
        print("Stack Underflow")
    else:
        print(f"Popped {stack.pop()}")

def peek():
    if stack:
        print(f"Top: {stack[-1]}")
    else:
        print("Stack is empty")

while True:
    print("\n1. Push 2. Pop 3. Peek 4. Exit")
    ch = input("Enter choice: ")
    if ch == '1': push(int(input("Item: ")))
    elif ch == '2': pop()
    elif ch == '3': peek()
    elif ch == '4': break
```

Output:

```
1. Push 2. Pop 3. Peek 4. Exit
Enter choice: 1
Item: 5
Pushed 5

1. Push 2. Pop 3. Peek 4. Exit
Enter choice: 3
Top: 5

1. Push 2. Pop 3. Peek 4. Exit
Enter choice: 1
Item: 76
Pushed 76

1. Push 2. Pop 3. Peek 4. Exit
Enter choice: 3
Top: 76

1. Push 2. Pop 3. Peek 4. Exit
Enter choice: 2
Popped 76

1. Push 2. Pop 3. Peek 4. Exit
Enter choice: 3
Top: 5

1. Push 2. Pop 3. Peek 4. Exit
Enter choice: 4
PS C:\Users\Aryan\OneDrive\Documents\College\Sem-2\DSA\Final PC File>
```

PC-02: Complexity Drill (Operation Counting)

Create 4 snippets: single loop, nested loop, triangular loop, and halving loop. Count operations roughly and map to Big-O. Add best/avg/worst reasoning for linear search and binary search.

```python
def counted_search(arr, target):
    ops = 0
    for i in range(len(arr)):
        ops += 1
        if arr[i] == target:
            print(f"Found at {i}. Total operations: {ops}")
            return
    print(f"Not found. Total operations: {ops}")

arr = [10, 20, 30, 40, 50]
print("Best Case (10):")
counted_search(arr, 10)
print("Worst Case (50):")
counted_search(arr, 50)
```

Output:

```
PS C:\Users\Aryan\OneDrive\Documents\College\Sem-2\DSA\Final PC File> &
Users/Aryan/OneDrive/Documents/College/Sem-2/DSA/Final PC File/count_ops
Best Case (10):
Found at 0. Total operations: 1
Worst Case (50):
Found at 4. Total operations: 5
PS C:\Users\Aryan\OneDrive\Documents\College\Sem-2\DSA\Final PC File>
```

PC-03: Recursive Factorial + Call Stack Trace

Implement factorial(n) recursively for n≥0 and reject invalid input. Draw call stack for factorial(4) showing return values.

```
def factorial(n, depth=0):
    indent = "  " * depth
    print(f"{indent}Calling factorial({n})")
    if n == 0 or n == 1:
        print(f"{indent}Returning 1")
        return 1
    res = n * factorial(n - 1, depth + 1)
    print(f"{indent}Returning {res}")
    return res

n = int(input("Enter number: "))
print(f"Result: {factorial(n)}")
```

Output:

```
PS C:\Users\Aryan\OneDrive\Documents\College\Sem-2\DSA\F
ers/Aryan/OneDrive/Documents/College/Sem-2/DSA/Final PC
Enter number: 5
Calling factorial(5)
  Calling factorial(4)
    Calling factorial(3)
      Calling factorial(2)
        Calling factorial(1)
        Returning 1
      Returning 2
    Returning 6
  Returning 24
Returning 120
Result: 120
PS C:\Users\Aryan\OneDrive\Documents\College\Sem-2\DSA\F
```

PC-04: Fibonacci (Naive + Memoized) + Call Counter

Implement fib naive(n) and fib memo(n) with call counters. Compare call counts for n=10,20,30 to show performance difference clearly.

```
naive_calls = 0
memo_calls = 0
memo = {}

def fib_naive(n):
    global naive_calls
    naive_calls += 1
    if n <= 1: return n
    return fib_naive(n-1) + fib_naive(n-2)

def fib_memo(n):
    global memo_calls
    memo_calls += 1
    if n in memo: return memo[n]
    if n <= 1: return n
    memo[n] = fib_memo(n-1) + fib_memo(n-2)
    return memo[n]

n = 10
print(f"Naive result: {fib_naive(n)}, Calls: {naive_calls}")
print(f"Memo result: {fib_memo(n)}, Calls: {memo_calls}")
```

Output:

```
Naive result: 55, Calls: 177
Memo result: 55, Calls: 19
PS C:\Users\Aryan\OneDrive\Documents\College\Sem-2\DSA\Final PC File>
                                                                Ln 21, C
```

PC-05: Tower of Hanoi

Implement Hanoi(n, src, aux, dst) and print exact move sequence for n=3. Show move count for n=4 and infer complexity.

```python
def hanoi(n, source, target, auxiliary):
    if n == 1:
        print(f"Move disk 1 from {source} to {target}")
        return
    hanoi(n-1, source, auxiliary, target)
    print(f"Move disk {n} from {source} to {target}")
    hanoi(n-1, auxiliary, target, source)

hanoi(3, 'A', 'C', 'B')
```

Output:

```
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
PS C:\Users\Aryan\OneDrive
```

PC-06: Recursive Binary Search + Recurrence Intuition

Implement binary search recursively binarySearch(arr, key, low, high) returning index or -1.

```
def binary_search(arr, low, high, x):
    if high >= low:
        mid = (high + low) // 2
        print(f"Checking mid-index {mid} (value: {arr[mid]})")
        if arr[mid] == x: return mid
        elif arr[mid] > x: return binary_search(arr, low, mid - 1, x)
        else: return binary_search(arr, mid + 1, high, x)
    return -1

arr = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
x = 23
print(f"Element {x} at index: {binary_search(arr, 0, len(arr)-1, x)}")
```

Output:

```
PS C:\Users\Aryan\OneDrive\Documents\College\Sem-
Users/Aryan/OneDrive/Documents/College/Sem-2/DSA/
Checking mid-index 4 (value: 16)
Checking mid-index 7 (value: 56)
Checking mid-index 5 (value: 23)
Element 23 at index: 5
PS C:\Users\Aryan\OneDrive\Documents\College\Sem-
```

Unit 2: Linear Data Structures

PC-07: Arrays 1D (Operations + Shifting Cost)

Perform insert at start/middle/end and delete at start/middle/end. For each case, compute and report approximate shift count.

```
arr = [10, 20, 30, 40, 50]
def insert_with_cost(val, pos):
    print(f"Inserting {val} at {pos}")
    shift_count = len(arr) - pos
    arr.insert(pos, val)
    print(f"Result: {arr}, Shifting operations: {shift_count}")

insert_with_cost(25, 2)
```

Output:

```
PS C:\Users\Aryan\OneDrive\Documents\College\Sem-2\DSA\Final
Users/Aryan/OneDrive/Documents/College/Sem-2/DSA/Final PC Fi
Inserting 25 at 2
Result: [10, 20, 25, 30, 40, 50], Shifting operations: 3
PS C:\Users\Aryan\OneDrive\Documents\College\Sem-2\DSA\Final
```

PC-08: Arrays 2D (Matrix Traversal + Ops)

Create a matrix and implement: row sum, column sum, search for a value, and transpose (conceptual output).

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
total = 0
for i in range(len(matrix)):
    for j in range(len(matrix[0])):
        print(f"Element at [{i}][{j}]: {matrix[i][j]}")
        total += matrix[i][j]
print(f"Matrix Sum: {total}")
```

Output:

```
PS C:\Users\Aryan\OneDrive\Docu
Users/Aryan/OneDrive/Documents/
Element at [0][0]: 1
Element at [0][1]: 2
Element at [0][2]: 3
Element at [1][0]: 4
Element at [1][1]: 5
Element at [1][2]: 6
Element at [2][0]: 7
Element at [2][1]: 8
Element at [2][2]: 9
Matrix Sum: 45
PS C:\Users\Aryan\OneDrive\Docu
```

PC-09: Dynamic Array Simulation (Resize + Pop)

Implement capacity/size based DynamicArray with append (doubling resize) and pop from end. Print (size, capacity) after each append.

```
import sys
dyn_arr = []
for i in range(10):
    dyn_arr.append(i)
    print(f"Size: {len(dyn_arr)}, Memory size: {sys.getsizeof(dyn_arr)} bytes")
```

Output:

```
Size: 1, Memory size: 88 bytes
Size: 2, Memory size: 88 bytes
Size: 3, Memory size: 88 bytes
Size: 4, Memory size: 88 bytes
Size: 5, Memory size: 120 bytes
Size: 6, Memory size: 120 bytes
Size: 7, Memory size: 120 bytes
Size: 8, Memory size: 120 bytes
Size: 9, Memory size: 184 bytes
Size: 10, Memory size: 184 bytes
PS C:\Users\Aryan\OneDrive\Documents\Colle
```

PC-10: Singly Linked List (Core Ops)

Implement dynamic insertion and deletion without shifting. Implement SLL with insert at beginning, insert at end, delete by value, traverse. Handle missing values safely.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class SLL:
    def __init__(self): self.head = None

    def insert(self, data):
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node

    def display(self):
        temp = self.head
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")

s = SLL()
s.insert(10); s.insert(20); s.display()
```

Output:

```
PS C:\Users\Aryan\One
Users/Aryan/OneDrive/
20 -> 10 -> None
PS C:\Users\Aryan\One
```

PC-11: Doubly Linked List (Extended Ops)

Implement DLL with insert after node(target,x) and delete at position(pos). Maintain prev/next correctly.

```
class DNode:
    def __init__(self, data):
        self.data = data
        self.next = self.prev = None

class DLL:
    def __init__(self): self.head = None
    def insert(self, data):
        nn = DNode(data)
        if self.head:
            self.head.prev = nn
            nn.next = self.head
        self.head = nn
    def traverse_back(self):
        t = self.head
        while t.next: t = t.next
        while t:
            print(t.data, end=" <-> ")
            t = t.prev
        print("None")

d = DLL()
d.insert(1); d.insert(2); d.traverse_back()
```

Output:

```
PS C:\Users\Aryan\OneDri
Users/Aryan/OneDrive/Doc
1 <-> 2 <-> None
PS C:\Users\Aryan\OneDri
```

PC-12: Stack using SLL + Parentheses Checker

Implement Stack (push/pop/peek) using SLL head as top. Use it to validate (), , [] in strings.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class Stack:
    def __init__(self):
        self.top = None  # head is top
    def push(self, data):
        new_node = Node(data)
        new_node.next = self.top
        self.top = new_node
    def pop(self):
        if self.top is None:
            return None
        popped = self.top.data
        self.top = self.top.next
        return popped
    def peek(self):
        if self.top is None:
            return None
        return self.top.data
    def is_empty(self):
        return self.top is None
def check_balanced(expr):
    stack = Stack()
    pairs = {')': '(', '}': '{', ']': '['}
    for char in expr:
        if char in '({[':
            stack.push(char)
        elif char in ')}]':
            if stack.is_empty():
                return False
            if stack.pop() != pairs[char]:
                return False
      return stack.is_empty()
tests = ["((1+2)*3)","{[()]}","{[(])}","(((","[]{}()","(]"]
for t in tests:
    print(f"{t} -> {check_balanced(t)}")
```

Output:

```
((1+2)*3) -> True
{[()]} -> True
{[(])} -> False
((( -> False
[]{}() -> True
(] -> False
PS C:\Users\Aryan\OneDrive\Do
```

PC-13: Queue using SLL (O(1) operations)

Implement queue with head+tail pointers and connect to BFS usage.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = self.prev = None

class Queue:
    def __init__(self): self.front = self.rear = None
    def enqueue(self, data):
        nn = Node(data)
        if not self.rear: self.front = self.rear = nn
        else:
            self.rear.next = nn
            self.rear = nn
    def dequeue(self):
        if not self.front: return "Empty"
        val = self.front.data
        self.front = self.front.next
        if not self.front: self.rear = None
        return val

q = Queue()
q.enqueue(100);q.enqueue(200); print(q.dequeue())
```

Output:

```
Users/Aryan/
100
PS C:\Users\
```

Unit 3: Sorting Algorithms

PC-14: $O(n^2)$ Sorts + Counting

Bubble Sort with swap counts.

```
def bubble_sort(arr):
    swaps = 0
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
                swaps += 1
    print(f"Swaps: {swaps}")
bubble_sort([64, 34, 25, 12])
```

Output:

```
Swaps: 6
PS C:\Users\Aryan\
```

PC-15: Insertion Sort (Advantage on Nearly Sorted)

Implement insertion sort and optionally print pass-wise array for a small input.

```
def insertion_sort(arr):
    comps = 0
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0:
            comps += 1
            if key < arr[j]:
                arr[j+1] = arr[j]
                j -= 1
            else: break
        arr[j+1] = key
    return comps

print(f"Comps (Sorted): {insertion_sort([1,2,3,4,5])}")
print(f"Comps (Reverse): {insertion_sort([5,4,3,2,1])}")
```

Output:

```
Comps (Sorted): 4
Comps (Reverse): 10
PS C:\Users\Aryan\One
```

PC-16: Merge Sort (Stable, O(n log n))

Implement merge(left,right) conceptually and merge sort. Note time O(n log n) and space O(n).

```python
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr)//2
        L, R = arr[:mid], arr[mid:]
        merge_sort(L); merge_sort(R)
        i = j = k = 0
        while i < len(L) and j < len(R):
            if L[i] <= R[j]: arr[k] = L[i]; i += 1
            else: arr[k] = R[j]; j += 1
            k += 1
        while i < len(L): arr[k] = L[i]; i += 1; k += 1
        while j < len(R): arr[k] = R[j]; j += 1; k += 1

arr = [38, 27, 43, 3, 9, 82, 10]
merge_sort(arr); print(arr)
```

Output:

```
PS C:\Users\Aryan\OneDrive\Doc
Users/Aryan/OneDrive/Documents
[3, 9, 10, 27, 38, 43, 82]
```

PC-17: Quick Sort (Partition + Worst Case)

Implement quick sort using a clear partition scheme (e.g., last pivot). Explain worst-case scenario (already sorted + bad pivot).

```
def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i+1], arr[high] = arr[high], arr[i+1]
    return i + 1

def quick_sort(arr, low, high):
    if low < high:
        pi = partition(arr, low, high)
        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)

arr = [10, 7, 8, 9, 1, 5]
quick_sort(arr, 0, len(arr)-1); print(arr)
```

Output:

```
PS C:\Users\Aryan\OneDri
Users/Aryan/OneDrive/Doc
[1, 5, 7, 8, 9, 10]
```

PC-18: Heap Sort (Heapify + Sorting)

Implement heapify + heap sort (min or max). Explain why heap sort is not stable.

```
def heapify(arr, n, i):
    largest = i
    l, r = 2 * i + 1, 2 * i + 2
    if l < n and arr[i] < arr[l]: largest = l
    if r < n and arr[largest] < arr[r]: largest = r
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)

def heap_sort(arr):
    n = len(arr)
    for i in range(n // 2 - 1, -1, -1): heapify(arr, n, i)
    for i in range(n-1, 0, -1):
        arr[i], arr[0] = arr[0], arr[i]
        heapify(arr, i, 0)

arr = [12, 11, 13, 5, 6, 7]
heap_sort(arr); print(arr)
```

Output:

```
PS C:\Users\Aryan\OneDriv
Users/Aryan/OneDrive/Docu
[5, 6, 7, 11, 12, 13]
```

PC-19: Benchmark Harness (Datasets + Timing Table)

Generate random/sorted/reverse datasets for sizes 1000/5000/10000. Time insertion, merge, quick and print a table (27 values).

```
import random, time
def insertion(a):
    a = a.copy()
    for i in range(1, len(a)):
        key, j = a[i], i-1
        while j >= 0 and a[j] > key:
            a[j+1] = a[j]
            j -= 1
        a[j+1] = key
    return a

def merge(a):
    if len(a) <= 1: return a
    m = len(a)//2
    l, r = merge(a[:m]), merge(a[m:])
    res, i, j = [], 0, 0
    while i < len(l) and j < len(r):
        if l[i] < r[j]: res.append(l[i]); i += 1
        else: res.append(r[j]); j += 1
    return res + l[i:] + r[j:]

def quick(a):
    if len(a) <= 1: return a
    p = a[len(a)//2]
    left = [x for x in a if x < p]
    mid  = [x for x in a if x == p]
    right= [x for x in a if x > p]
    return quick(left) + mid + quick(right)

def gen(n, case, seed):
    random.seed(seed)
    if case == "random": return [random.randint(1, 100000) for _ in range(n)]
    if case == "sorted": return list(range(n))
    return list(range(n, 0, -1))  # reverse
def run():
    sizes = [1000, 5000, 10000]
    cases = ["random", "sorted", "reverse"]
    algos = {"Insertion": insertion, "Merge": merge, "Quick": quick}

    print(f"{'Size':<6}{'Case':<10}{'Algo':<10}{'Time(s)':<10}")
    print("-"*36)

    for n in sizes:
```

```
        for c in cases:
            data = gen(n, c, 42)
            for name, f in algos.items():
                t1 = time.perf_counter()
                f(data)
                t2 = time.perf_counter()
                print(f"{n:<6}{c:<10}{name:<10}{(t2-t1):<10.5f}")

run()
```

Output:

```
Size  Case      Algo      Time(s)
-----------------------------------
1000  random    Insertion 0.02168
1000  random    Merge     0.00206
1000  random    Quick     0.00455
1000  sorted    Insertion 0.00016
1000  sorted    Merge     0.00537
1000  sorted    Quick     0.00215
1000  reverse   Insertion 0.04881
1000  reverse   Merge     0.00147
1000  reverse   Quick     0.00137
5000  random    Insertion 0.58713
5000  random    Merge     0.01115
5000  random    Quick     0.02067
5000  sorted    Insertion 0.00054
5000  sorted    Merge     0.00815
5000  sorted    Quick     0.01573
5000  reverse   Insertion 2.18835
5000  reverse   Merge     0.01514
5000  reverse   Quick     0.02205
10000 random    Insertion 2.06864
10000 random    Merge     0.03085
10000 random    Quick     0.02461
10000 sorted    Insertion 0.00114
10000 sorted    Merge     0.01753
10000 sorted    Quick     0.02318
10000 reverse   Insertion 4.43602
10000 reverse   Merge     0.01834
10000 reverse   Quick     0.02306
PS C:\Users\Aryan\OneDrive\Documen
```

Unit 4: Non-Linear & Advanced Data Structures

PC-20: BST Insert/Search/Inorder

Implement BST insert, search, inorder traversal. Use a fixed dataset and print inorder result.

```
class BSTNode:
    def __init__(self, key):
        self.left = self.right = None
        self.val = key

def insert(root, key):
    if root is None: return BSTNode(key)
    if key < root.val: root.left = insert(root.left, key)
    else: root.right = insert(root.right, key)
    return root

def inorder(root):
    if root:
        inorder(root.left)
        print(root.val, end=" ")
        inorder(root.right)

r = insert(None, 50); insert(r, 30); insert(r, 70); inorder(r)
```

Output:

```
Users/Aryan/
30 50 70
PS C:\Users\
```

PC-21: BST Delete (All Cases)

Implement delete handling: leaf, one child, two children (successor). Print inorder after each delete as proof.

```
class Node:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None
def insert(root, key):
    if root is None:
        return Node(key)
    if key < root.val:
        root.left = insert(root.left, key)
    else:
        root.right = insert(root.right, key)
    return root
def inorder(root):
    if root:
        inorder(root.left)
        print(root.val, end=" ")
        inorder(root.right)
def min_value_node(node):
    curr = node
    while curr.left:
        curr = curr.left
    return curr
def delete_node(root, key):
    if root is None:
        return root

    if key < root.val:
        root.left = delete_node(root.left, key)

    elif key > root.val:
        root.right = delete_node(root.right, key)

    else:
        # Case 1: Leaf node
        if root.left is None and root.right is None:
            return None

        # Case 2: One child
        elif root.left is None:
            return root.right
        elif root.right is None:
            return root.left

        # Case 3: Two children
        temp = min_value_node(root.right)  # inorder successor
```

```python
        root.val = temp.val
        root.right = delete_node(root.right, temp.val)

    return root
# Build BST
root = None
values = [50, 30, 70, 20, 40, 60, 80]

for v in values:
    root = insert(root, v)

print("Initial Inorder:")
inorder(root)
print("\n")

# Deletions
delete_keys = [20, 30, 50]

for key in delete_keys:
    print(f"Deleting {key}...")
    root = delete_node(root, key)
    print("Inorder after deletion:", end=" ")
    inorder(root)
    print("\n")
```

Output:

```
Initial Inorder:
20 30 40 50 60 70 80

Deleting 20...
Inorder after deletion: 30 40 50 60 70 80

Deleting 30...
Inorder after deletion: 40 50 60 70 80

Deleting 50...
Inorder after deletion: 40 60 70 80
```

PC-22: Heap / Priority Queue (Industry)

Min-heap or max-heap with insert, peek, extract-min/max. Show extraction order to verify correctness.

```
import heapq
pq = []
heapq.heappush(pq, (2, "Task B"))
heapq.heappush(pq, (1, "Task A"))
heapq.heappush(pq, (3, "Task C"))
while pq:
    print(heapq.heappop(pq))
```

Output:

```
(1, 'Task A')
(2, 'Task B')
(3, 'Task C')
```

PC-23: Graph Build (Adjacency List)

Create 5–7 nodes and 8–10 edges. Store as adjacency list and print readable structure.

```
graph = {}
def add_edge(u, v, w):
    if u not in graph: graph[u] = []
    graph[u].append((v, w))

add_edge('A', 'B', 5); add_edge('B', 'C', 3)
print(graph)
```

Output"

```
{'A': [('B', 5)], 'B': [('C', 3)]}
PS C:\Users\Aryan\OneDrive\Documents\Co
```

PC-24: BFS Traversal

Implement BFS(start) using queue and visited set. Print BFS order from a chosen start node.

```
graph = {}
def add_edge(u, v, w):
    if u not in graph: graph[u] = []
    graph[u].append((v, w))

add_edge('A', 'B', 5); add_edge('B', 'C', 3)

def bfs(graph, start):
    visited = [start]
    queue = [start]
    while queue:
        u = queue.pop(0)
        print(u, end=" ")
        for v in graph.get(u, []):
            if v[0] not in visited:
                visited.append(v[0])
                queue.append(v[0])

bfs(graph, 'A')
```

Output:

```
e "C:/Us
A B C
PS C:\Us
```

PC-25: DFS Traversal

Implement DFS(start) recursively or using stack. Print DFS order from start node.

```
graph = {}
def add_edge(u, v, w):
    if u not in graph: graph[u] = []
    graph[u].append((v, w))

add_edge('A', 'B', 5); add_edge('B', 'C', 3)

visited = set()
def dfs(graph, node):
    if node not in visited:
        print(node, end=" ")
        visited.add(node)
        for neighbor in graph.get(node, []):
            dfs(graph, neighbor[0])

dfs(graph, 'A')
```

Output:

```
e "c:/Us
A B C
PS C:\Us
```

PC-26: Hash Table (Separate Chaining)

Implement insert/get/delete with chaining. Use key % tableSize hash. Demonstrate collisions with multiple keys.

```
class HashTable:
    def __init__(self, size):
        self.size = size
        self.table = [[] for _ in range(size)]
    def hash_fn(self, key): return key % self.size
    def insert(self, key, val):
        idx = self.hash_fn(key)
        self.table[idx].append((key, val))
    def get(self, key):
        idx = self.hash_fn(key)
        for k, v in self.table[idx]:
            if k == key: return v
        return None

ht = HashTable(5); ht.insert(10, "Ten"); print(ht.get(10))
```

Output:

```
Ten
PS C:\U
```

PC-27: Trie (Prefix Tree)

Implement trie insert/search/startsWith. Use a small word set and test prefix searches.

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False

class Trie:
    def __init__(self): self.root = TrieNode()
    def insert(self, word):
        node = self.root
        for char in word:
            if char not in node.children: node.children[char] = TrieNode()
            node = node.children[char]
        node.is_end = True
    def search(self, word):
        node = self.root
        for char in word:
            if char not in node.children: return False
            node = node.children[char]
        return node.is_end

t = Trie(); t.insert("apple"); print(t.search("apple"))
```

Output:

```
e "c:/Use
True
PS C:\Use
```

PC-28: Bloom Filter (Concept + Toy Demo)

Explain bloom filter (bit array + k hash functions). Demonstrate toy membership test and false positive concept.

```
class BloomFilter:
    def __init__(self, size):
        self.bit_array = [0] * size
    def add(self, item):
        idx1 = hash(item) % len(self.bit_array)
        idx2 = (hash(item) // 7) % len(self.bit_array)
        self.bit_array[idx1] = 1
        self.bit_array[idx2] = 1
    def check(self, item):
        idx1 = hash(item) % len(self.bit_array)
        idx2 = (hash(item) // 7) % len(self.bit_array)
        return self.bit_array[idx1] == 1 and self.bit_array[idx2] == 1

bf = BloomFilter(20); bf.add("test"); print(bf.check("test"))
```

Output:

```
e "c:/Use
True
PS C:\Use
```